One bounded learning experiment

M21 • 01 Aug 2028 - 03 Sep 2028 • 75 hours

What you will achieve this month

The deliverable is learning_eval_v1: a comparison of imitation learning and a small PPO experiment against unchanged classical control. The report includes tests under conditions absent from training and verification of the return to the baseline algorithm. The main task is a small correction to the body orientation of a standing quadruped within specified ranges of tilt and disturbance. The learned component produces only a bounded high-level correction; the actuator loop, contact constraints and independent monitor remain classical.

In 75 hours, study demonstration collection, behaviour cloning, MDPs, rewards, PPO, random-number generator initialisation (seed), environment parameter randomisation and a safety filter. Learning a complete dynamic gait from scratch, jumping and transfer to real actuators are outside this module. All imitation learning (IL) and reinforcement learning (RL) experiments here take place in simulation. An increase in reward alone does not justify running a new policy on hardware.

Prerequisites

From M19-M20, you need stable standing (stand) or a slow-gait pause mode (pause), a verified QP or conservative force-control mode, the actual model limits, and independent detection of stale state data, loss of support and unacceptable commands. From M18, bring data-handling rules, grouped dataset splits, a small network, an experiment log and a CPU workload profile. Save the working baseline with its commit identifier and configuration; it must remain unchanged in the comparison with ML.

If classical posture holding fails its own checks, investigate that defect first. Check the data, describe the MDP and train on the valid demonstrations available; postpone closed-loop trials with the learned policy until the cause is resolved. Until verification is complete, learning remains an optional branch and hardware use is prohibited. G3, moved into the overall calendar, remains a separate integration checkpoint and does not replace approval to conduct the experiment.

Connections to your projects

For the walking robot, the result will show whether a small learned correction can improve body recovery under bounded uncertainty. For the hand, the equivalent is a bounded correction to position, orientation or grip force added to the command of a working classical controller. Do not train both systems this month: describe how the formulation would transfer to a finger as a separate exercise, specifying contact features and the different meaning of failure.

A successful outcome may be that the baseline is already adequate and ML does not improve the selected metrics. This conclusion is acceptable when comparison conditions are identical and results are reproducible. A new policy does not have to replace the verified algorithm; measurements must establish its usefulness and its limitations must remain explicit.

Time, computing resources and experiment limits

The 75-hour plan

Hours 1-20: 10 hours of theory and 10 of practice. Study demonstration datasets, behaviour cloning, error accumulation, shifts in the distribution of visited states, the DAgger concept and data auditing. Prepare a small demonstration dataset split by episode and a first imitation model. Evaluate it on recorded data and in closed-loop operation. The baseline controller must already work: redeveloping it is outside this allocation.

Hours 21-40: 10 hours of theory and 10 of practice. Study MDPs, observations, actions, rewards, episode termination, PPO, learning curves and variation across seeds. Prepare a valid environment and complete three small, reproducible runs. For SAC, understanding its differences from PPO is sufficient; implementing a second RL algorithm is not required.

Hours 41-60: 10 hours of theory and 10 of practice. Examine the effects of mass, friction, delay, noise and terrain, progressively harder training and randomisation limits. Prepare separate training and evaluation profiles, test each parameter family and conduct bounded ablations. Hours 61-75: 5 hours of theory and 10 of practice. Study OOD, the safety filter, independent constraints, fallback operation and recovery after a push. Compile the final test table and a protocol for future transfer from simulation to hardware.

A realistic CPU budget

The default platform is a CPU and the existing simulator running without graphics. Start with one or two small network layers and two actions: roll and pitch corrections. For each of the three main PPO runs with different seeds, set an upper limit of 50 000 policy steps or 30 minutes of training, whichever comes first; this is an educational limit, not a convergence guarantee. Record actual steps and time. If the environment is too slow, reduce the step count equally for all seeds, retain all three runs and explain how the limit constrains your conclusions.

For the five short ablations, use one seed selected in advance and a maximum of 10 minutes per variant. This analysis is preliminary: it does not replace the three-seed comparison for the main result. Total machine time for these training runs is bounded; while they execute, analyse earlier logs within the same practice block without counting the same time twice. No GPU is assumed. Apple M4 is not a CUDA platform; consider Isaac Lab separately only with compatible hardware.

Placement in the calendar

Theory: five 2-hour sessions in each of the first three blocks, then 2 + 2 + 1. Practice: four 10-hour blocks, with breaks accounted for separately. Date boundaries may coincide with the adjacent module; retain rest, holidays and integration blocks from the overall plan. Hour numbers define the work sequence; they do not create a new calendar or imply double-counting time.

Demonstrations: what the model should learn

Two roles for the learned component

In behaviour cloning (BC), the expert is an unchanged classical function: it selects a bounded target correction to body orientation from the observation. Choose scenarios with small, non-zero tilts and disturbances so that actions are not almost always zero. The low-level controller from M20 tracks this target. PPO later uses a separate residual-correction mode: an even smaller learned correction is added to the classical command. Label the modes separately and do not combine them into one metric.

Task 1. Audit the expert and data

Collect independent standing episodes with small deviations and disturbances known in advance. Record observation, expert_action, applied_action, contacts, data age, scenario identifier, seed and termination reason. Save the true simulator parameters and ground-truth state for analysis, but do not expose them to the policy if the future robot cannot measure them. List every expert input: explicitly identify any hidden privileged information.

Split whole episodes into training (train), validation and final test sets. All windows from one trial must belong to the same set; calculate normalisation parameters from training data only. Check the action and state distributions, saturation fraction, erroneous commands and episodes containing failures. Do not silently discard invalid demonstrations: specify the exclusion rule before training and retain the exclusion count. Exclude samples following a physically invalid environment reset from training.

Task 2. Behaviour cloning

Train a compact network to minimise the mean squared deviation from the expert correction in normalised units. For angles, also report MAE in radians or degrees, labelled explicitly. Start with the simplest method, the mean action in the training set, and a linear model; compare the neural network against them on the same dataset. Check large expert corrections separately: average error can hide poor behaviour precisely when conditions become difficult.

Save weights, normalisation parameters, feature list, configuration and predictions. For verification, replay a separate episode and plot expert and model actions. Boundary cases are a zero command, an unknown schema, a stale observation and an action at its limit. Pass only independently checked actions to the controller; replacing NaN with an admissible zero must be recorded in the log.

The first step aims to reproduce a bounded decision from available observations, rather than establish superiority over the expert. If the expert uses information unavailable to the model, some error may be unavoidable; identify this before increasing network size.

Error accumulation and the DAgger concept

Why error on recorded data is insufficient

In recorded data, the model receives states visited by the expert. When it controls the system itself, even a small action deviation changes the next state; it then encounters less familiar states and makes further errors. Low error on demonstrations therefore does not guarantee stable closed-loop control. Compare commands, visited states, safety interventions and episode outcomes.

Task 3. Evaluation on recordings and in closed-loop operation

First replay test-set observations and calculate action error without allowing the model to influence simulation. Then, in a separate evaluation environment, let BC select the high-level correction through the safety filter, preserving the same initial conditions. Compare body trajectories, maximum tilt, recovery time, command clamping and rejection rates (clamp and reject), and falls. Filter parameters must be identical for all controllers being compared.

Choose one episode in which error grows gradually. Locate the first substantial divergence and compare the observation against the training range. Do not attribute everything to OOD: check delay, normalisation, privileged expert features and the actual sensitivity of the dynamics to the action. Save a plot of the causal chain: command error → state change → next error.

Task 4. A minimal DAgger iteration

Study the principle: collect states visited by the current policy, query the expert for actions in those states and add the pairs to the training set. One bounded iteration in simulation is sufficient for this educational experiment. The expert labels new states while the independent filter supervises execution. Do not continue an episode beyond admissible states simply to diversify the data.

Add new episodes only to the training set; do not use the validation or final test sets for training. Retrain by the same method and compare results on a separate diagnostic set of control episodes. Show changes in state distribution, action errors and intervention counts. Record a lack of improvement if that is the result. One iteration does not establish all theoretical properties of DAgger; unlimited data collection is unnecessary here.

Transferring the concept to the hand

Consider a finger pressing against an object: a small error can move the object, causing subsequent states to diverge from the demonstration. Describe available measurements, the expert command and the condition for ending contact. Complete this written exercise within the analysis allocation; do not launch a second learning project. For both robots, assess closed-loop behaviour separately from imitation accuracy along another controller's trajectory.

Freeze BC as a separate version. When moving to PPO, retain its results: imitation of an expert and improvement through a residual correction solve different problems, and both sets of results must remain available for inspection.

An MDP and an environment with explicit actions

The learning-task contract

An MDP comprises a state, action, transition, reward and termination rule. The real agent receives an observation that may omit parts of the state, creating partial observability. For the small experiment, use available roll and pitch, angular velocities, desired-pose errors, contact statuses, measurement age and previous action. The exact friction coefficient and a future push are not observation inputs.

Task 5. A Gymnasium environment

Implement reset(seed) and step(action) around the existing simulator. The action comprises two normalised values in $[-1,1]$, scaled to a small residual roll and pitch correction added to the classical command. Derive the maximum correction and its rate of change from the previously verified simulation range. The policy does not control currents directly. The low-level controller and monitor retain their existing limits.

Follow this step order: check observation freshness, calculate the raw action, apply the filter, execute several physics steps with the filtered command, then obtain the new observation, reward and termination flag. Save raw and applied actions separately. If the filter frequently replaces an action, both policy and protection determine the result; reflect this in the metrics. Exceeding numerical limits and falling terminate the episode under the task conditions; label an administrative time limit truncated.

Task 6. Reward and undesirable strategies

Start with a negative weighted sum of squared body-orientation error, angular velocity, residual magnitude and action change. Normalise quantities by declared characteristic scales so that weights have a clear meaning. If the reward approximates an integral cost, account for dt consistently when changing frequency. Add a penalty for violating conditions, but do not expect it to replace a hard safety filter.

Manually check four sequences: holding posture, an oscillating correction, rapid episode termination and passive waiting. Calculate the reward components. An early fall must not appear advantageous merely because it stops negative costs accumulating. Save reward components and a separate success flag based on engineering criteria. Increased return without reduced tilt or fewer failures is not an improvement to the robot.

Check reset: the same seed reproduces initial conditions, different seeds actually change the declared parameters, and filter state and previous command are cleared. State leakage between episodes can create a hidden dependence on test order.

PPO: a small reproducible experiment

What to understand about the algorithm

PPO updates a policy from environment interactions while limiting changes in the objective. Study trajectories, returns, discounting, the value function, action advantage and clipping of the probability ratio. Use Stable-Baselines3 and a simple multilayer perceptron (MLP). Full proofs and a custom RL framework are unnecessary; understand observations, actions and episode termination before running training.

Task 7. Check the environment and run the first trial

First run the automated environment API check, trials with random admissible actions and trials with zero residual correction. With zero correction, behaviour must match the frozen baseline using the same low-level controller. Then run a very short training session to check implementation: rewards remain finite, episodes end correctly, monitor weights remain unchanged and the step counter advances. Fix environment errors before tuning PPO.

Freeze the rollout step count, batch size, learning rate, discount factor and total step limit. Start from the configuration in the documentation, adapting only what is necessary for your episode frequency and duration; explain every change. Do not run dozens of variants and select the best on test disturbances. One selected PPO experiment matches the agreed scope of the month.

Task 8. Three runs and reproducible curves

Complete three independent runs with seeds listed in advance, identical limits and separate directories. Three evaluations of one trained network do not count as three independent training runs. Save learning curves, checkpoints and timing logs for each. For a model stopped by the time limit, state the actual number of interactions: a faster run may have seen more data. For a strict comparison by step count, use a common achievable limit.

After training, evaluate all three policies on the same list of independent episodes. Report each result, the aggregate mean and spread, and the result for the worst-performing seed. Do not show only the successful instance in the final plot. Select checkpoints using validation data; use the final test set after the choice is frozen. If all models are worse than the baseline, this is a valid negative result for the bounded experiment.

When to stop training

The limit is 50 000 steps or 30 minutes per seed; reduce it equally when physics is slow. This budget may be insufficient to produce improvement; state that explicitly. Do not compensate for limited time by disabling the monitor or supplying ground-truth state as observations without declaring a different formulation. Check environment quality, action scale and reward first; allocate a new computing budget in a future experiment.

The required branch runs on CPU. Decide whether to use an accelerator after measuring runtime costs: a small MLP may take less time than the physics calculation. Isaac Lab and large batches of GPU environments are a separate choice and are not required to complete M21.

Parameter randomisation with physical meaning

Five families of uncertainty

Mass and inertia change the required forces; friction affects contact feasibility; delay changes when a command takes effect; sensor noise and bias distort observations; terrain height and slope change support. Do not choose arbitrary ranges without a stability rationale. For each family, record the nominal value, training and validation ranges, a separate OOD range, units and source: measurement, specification or an assumed range for the educational simulation.

Task 9. A randomisation register

Sample mass and friction at environment reset, and noise according to an explicit temporal model; delay may remain constant within an episode or vary by a declared rule. Save the actual sampled parameters for each episode. Keep inertia consistent when changing mass, and avoid negative friction or impossible geometry. Mark physically invalid examples as invalid simulation; do not count them as valid OOD tests.

Start with a narrow range that the classical baseline can handle. Increase training difficulty progressively when a predefined criterion is met, such as a sufficient fraction of validation episodes without falls or excessive interventions. Choose the threshold for the educational scenario before running it. Progression must not depend on the final test set; save stage durations and boundaries in the configuration.

Task 10. Parameter-family ablations

For the main result, train PPO with the chosen randomisations. Then run five short preliminary variants, disabling mass, friction, delay, noise and terrain randomisation in turn while retaining all other conditions and one seed selected in advance. The budget is up to 10 minutes per variant. Evaluate them on the same diagnostic dataset for the relevant family. This is a limited training ablation; one seed cannot establish a robust causal effect.

Separately evaluate the trained model while varying parameters systematically: change one family at a time and hold the others at nominal values. Plot failures and errors against the magnitude of change. The two experiments answer different questions: whether randomisation is needed during training, and how the finished policy responds to changed conditions. Varying evaluation parameters alone does not demonstrate the effect of training randomisation.

Limits of transfer from simulation to hardware

Randomisation broadens familiar conditions, but does not guarantee coverage of every hardware condition. Backlash, elasticity, heating, communication delays and contact errors may be absent from the model. List these discrepancies and how to measure them later. For the hand, object friction, finger compliance and tactile-sensor properties would also matter; renaming variables cannot transfer the current body policy to that task.

Independent monitoring, OOD and fallback control

Separate policy evaluation from authorisation to execute

A policy shield checks or limits the proposed correction against known conditions. It is not trained with PPO and receives no reward for passing an unsafe command. Check finite values, amplitude, rate of change, observation age, pose, contacts and available limits throughout the actuation chain. The QP and M20 controller retain their constraints on the complete command, including torque and modelled current.

Task 11. An OOD indicator and its reliability limits

Start with simple checks on standardised feature ranges and distance from training data. Choose thresholds using validation data, then freeze them. Document that this indicator misses some unfamiliar states and may reject familiar ones. Action probability, PPO policy entropy and network confidence do not guarantee physical reliability. Measure false alarms and undetected bad episodes for each OOD scenario.

Inject test faults: sensor bias, a delayed observation, a new slope, low friction and a missing foot. Record the first unreliable input, monitor activation and transition to fallback. Calculate response latency on one time base. Also test a new combination of conditions whose individual values all lie within admissible numerical ranges: componentwise checks may miss this combination.

Task 12. Transition to fallback control

When a command is rejected (reject), disable the learned residual according to a predefined rule; classical standing (stand) or gait pause remains available. During ordinary deactivation, the correction may decrease smoothly within admissible limits. When a critical constraint is violated, protection takes priority over smoothness. Do not continue applying a stale learned action for a few more samples after its validity period expires.

In BC mode, fallback restores the expert high-level command; in residual PPO mode, it zeros the correction added to baseline control. Both modes use the same low-level controller. Resuming ML requires reliable data for a specified duration and an explicit state-machine condition; one normal sample is insufficient. Record the fraction of time under baseline control: if protection almost constantly replaces the policy, successful episodes cannot all be credited to it.

Force a fallback transition while the body is displaced and compare it with deactivation during quiet standing. Show raw and applied actions, state, reason and constraints. The ability to fall back does not prove that every already dangerous state is recoverable; outside the verified region, record a failure rather than a guaranteed recovery.

Integrated evaluation: learning_eval_v1

Comparison modes

Compare four explicitly labelled modes: classical baseline control; cloning of the high-level command; baseline control with a PPO residual; and the same PPO with mandatory monitoring and fallback. If unprotected PPO is needed for diagnosis, run it only in simulation with external stopping conditions; that experiment does not authorise hardware use. BC and residual PPO solve different tasks, so action-prediction error is not a common measure of control quality.

The primary candidate for further use is the protected scheme. For a fair comparison, the baseline uses the same sensors, delays, constraints and scenarios. Evaluation shares initial states, disturbances and seeds; list training seeds separately. If the baseline is modified after ML fails, repeat the complete comparison and explicitly identify its new version.

Task 13. Compare recovery after a push

Apply a small reproducible impulse at a known body location and episode time, within the previously verified simulation range. Vary direction and intensity on a fixed grid. For each mode, measure maximum tilt, time to return to a declared band and remain there for the required duration, RMS error, saturations, command clamps and rejections, falls and successful completion. Retain episodes without a push as well: the policy must not degrade ordinary standing in exchange for an occasional successful recovery.

Report reward in a separate column. If reward rises while interventions or falls increase, reconsider the objective instead of claiming success. State the episode count in each cell and show all three PPO runs with different seeds. Variation across seeds does not replace statistics across diverse scenarios; show both sources of uncertainty.

Task 14. A hardware-transfer protocol without operating actuators

Prepare the sequence of future checks: unit and schema verification; processing real recordings without control; shadow operation without affecting the robot; a constrained test rig with known loads; independent limit and stopping checks; only then consideration of physical use. Specify required data and progression criteria for each step. M21 produces the protocol; it does not perform automatic deployment.

Allowable current, temperature and torque, together with mechanical limits, must come from the specific actuators and earlier-stage tests. Missing values remain unresolved dependencies. The separate G3 includes data, model and deployment audits and a decision on Jetson and learned motion control; moving its date does not permit bypassing checks. The M21 CPU path does not depend on buying an accelerator. Pass a working classical algorithm to M22 whether or not learning proved useful.

Session-by-session schedule: 1-75

Block 1: hours 1-20

1-2: verify the baseline and available observations. 3-4: imitation learning and the action loss function. 5-6: grouped data splitting and demonstration labelling. 7-8: error accumulation and state-distribution shift. 9-10: the DAgger concept and control-episode protocol. Practice 11-20: 2 hours each for data collection and auditing, dataset splitting and normalisation, BC training, closed-loop evaluation, and one bounded data-aggregation iteration. Save results before moving to RL.

Block 2: hours 21-40

21-22: MDPs and partial observability. 23-24: observation, action and policy frequency. 25-26: reward, task termination and truncation by an external limit. 27-28: return, action advantage and PPO clipping. 29-30: seeds, checkpoint selection and independent evaluation. Practice 31-40: 2 hours for the environment and reset; 2 for reward and zero correction; 2 for a short PPO check and logging setup; 2 for three bounded runs with concurrent log analysis; 2 for comparison and archiving.

Block 3: hours 41-60

41-42: physical meaning of mass, inertia and friction. 43-44: delays, noise and temporal correlation. 45-46: terrain, training ranges and OOD. 47-48: progressively harder training and acceptance limits. 49-50: the distinction between training ablation and evaluation parameter sweeps. Practice 51-60: 2 hours for the parameter register; 2 for the main randomised configuration; 2 for bounded ablations; 2 for parameter-family sweeps; 2 for failure analysis. When retraining the main randomised variant, retain three seeds and the same per-run limit; this is included in this block.

Block 4: hours 61-75

61-62: OOD indicators and their reliability limits. 63-64: the independent safety filter, complete-command constraints and fallback. Hour 65: the written hardware-transfer protocol and decision criteria. Practice 66-75: 2 hours for monitoring and input-fault injection; 2 for fallback transitions; 2 for the common push test; 2 for evaluating all models and seeds; 2 for reproduction from a clean directory and the final report.

What to save after each session

Each theory session ends with a diagram, a worked example or a decision on interface requirements. Each practice session ends with a configuration, raw log, numerical metric, failure case and next question. Do not extend training without new checks: the learning objective is to understand why a policy works or fails within the agreed budget. Queueing, loading and waiting must not consume the time needed to analyse sensors, constraints and comparison quality.

Acceptance: eight checks and the final decision

Required scenarios

1. Zero residual correction. Behaviour matches the baseline exactly; any difference indicates a wrapper error. **2. Faulty reset.** Repeat the seed and verify that previous actions, filters and timers are cleared. The test must detect hidden memory between episodes.

3. Invalid observation. Supply NaN, an incorrect schema and expired data. The monitor rejects the action and logs the reason. **4. Action at the limit.** Supply a large step in policy output. Check amplitude, rate of change and the complete command after the low-level controller; report every command limitation.

5. OOD friction or mass. Move one parameter family beyond the training range while preserving physical validity. Measure errors, falls and OOD detection; high network confidence does not exclude failure. **6. A combination of familiar extremes.** Combine delay, noise and slope within their individual ranges. Show whether a simple detector can miss a dangerous combination.

7. Forced fallback. Disable the policy during quiet and displaced standing. Measure the command jump and recovery; check that automatic reactivation does not occur. **8. An identical push.** Run the common episode list for the baseline, BC and three PPO runs with different seeds. Retain poor results and compare engineering metrics against reward.

Completion criteria

Prepare an unchanged baseline version, a demonstration description, dataset splits without leakage, a BC model, a verified environment and bounded PPO runs with at least three seeds, or a documented reason for stopping before training. Save configurations, normalisation, checkpoints, actual steps and runtime, OOD profiles, ablations and the fallback report. If baseline or monitor checks fail, use of the learned policy is prohibited; state this explicitly in the deliverable.

The final decision selects one outcome: retain the baseline; continue research in simulation; or prepare a separate next verification stage. This table does not automatically authorise a physical run. Claim improvement only against metrics selected in advance, without degrading mandatory constraints. For a policy requiring frequent interventions, state separately how much success is provided by the classical loop.

Self-check and scope reduction

Explain the distinctions between recorded-data and closed-loop evaluation, BC and a residual correction, state and observation, terminated and truncated, reward and success, training randomisation and condition shifts during evaluation. Show the expert's hidden information, seed selection, safety-filter operation and OOD-detector limitations. If time is short, omit Isaac Lab, new architectures and extended curriculum learning. Retain the small CPU environment, independent runs, classical comparison, OOD and fallback. No improvement is an acceptable outcome; work without verification is incomplete.

Assigned reading: imitation and one RL experiment

The references may be in English; the exercises and definitions are presented here in English. Read only the named sections before the associated work, within the existing 75 hours.

Core reading: three sources

1. R. Sutton, A. Barto - Reinforcement Learning: An Introduction, 2nd ed. Publisher's contents. Sections 3.1-3.5: interaction, reward, episode, policy and value; 13.1, 13.3-13.5: policy gradient, REINFORCE and actor-critic. For tasks 5-8, write out your environment's MDP and the meaning of action advantage; reading the whole book is unnecessary.

2. S. Levine - CS 185/285, Supervised Learning of Behaviors. Lecture 2, parts 2, 3 and 5: the BC algorithm, distribution shift and DAgger. Before tasks 1-4, explain why error on recorded frames differs from error during the model's own closed-loop control.

3. R. Tedrake - Underactuated Robotics, MIT notes. Model-Free Policy Search → Policy Gradient Methods. Selected reading: REINFORCE and Sample efficiency. Before the three runs with different seeds, estimate the cost of one experiment and explain why a small CPU budget limits conclusions rather than proving that RL fails.

Reference sections for practical tasks

Stable-Baselines3. PPO: Notes, Example, Parameters and RL Tips: Evaluation, Custom environments. For tasks 7-8: one pinned implementation, identical comparison scenarios and multiple seeds.

Gymnasium. Env: reset, step, spaces and Handling Time Limits. For task 5: the environment contract, the distinction between terminal and truncated, and correct estimation of future return.

Optional: original papers

Ross et al.: DAgger, the aggregation algorithm; Schulman et al.: PPO, clipped objective; Tobin et al.: Domain Randomization. The last paper mainly concerns visual transfer; ranges for mass, friction and delay need a separate rationale.

Reading deliverable: one page explaining the MDP, the distinction between BC and PPO residual correction, and the criteria for returning to the frozen controller.

Terms and definitions

Behaviour cloning (BC). Learning a mapping from observations to actions using labelled expert examples and an ordinary prediction-error loss.

DAgger. Iteratively adding states visited by the current policy to the training set, with actions freshly labelled by the expert in those states.

MDP. A model of sequential decisions comprising state, action, probabilistic transition and reward; the next state depends on the current state and action.

Observation. Information available to the agent at a step; it may omit part of the physical state, so the input may lack the Markov property.

Policy. A rule for choosing an action from available information; a stochastic policy defines a probability distribution over admissible actions.

Reward. A number received for one transition; it expresses the selected learning objective but is not by itself an engineering success criterion.

Return. The sum of future rewards with specified discounting from a chosen point in the episode; a random variable until the trajectory is observed.

Discount factor γ. A dimensionless multiplier of future rewards; values of γ closer to one increase the relative contribution of more distant consequences.

State value V. Expected return when starting in a given state and subsequently following the specified policy in the given environment.

Advantage. The difference between the expected value of a particular action Q(s,a) and the average state value V(s); it measures the action's relative usefulness.

PPO. A policy-update algorithm that clips the probability ratio in a surrogate objective; this moderates changes but does not guarantee safe motion.

Residual policy. A bounded learned addition to a controller command; its scale, rate of change and deactivation are specified separately.

Rollout. A sequence of policy-environment interactions: observations, actions, rewards and terminations; distinct from replaying a fixed recording.

Termination and truncation. Ending because a task condition is met and interruption by an external limit affect estimation of future return differently.

Distribution shift. A change in the frequency or region of observed states relative to the training set, including effects of the agent's own incorrect actions.

Domain randomization. Sampling simulation parameters from specified distributions during training; the ranges match the experiment's physical assumptions.

OOD. Conditions outside the established training distribution; a detector may miss them, so independent constraints remain in force.

Fallback. A verified transition to a classical mode or stop when the learned component fails; it needs its own switching condition and test.

Relationships: $G_t = r_{t+1} + \gamma G_{t+1}$; $A(s,a) = Q(s,a) - V(s)$. Check: high reward and low BC loss do not yet establish stable behaviour.

Equipment, purchases and installation

USE EXISTING EQUIPMENT

Use the MacBook Pro M4, the M18 CPU environment, the M19-M20 model and simulator, the unchanged classical baseline and experiment logs. Physical actuators do not participate in training episodes; the MCU, IMU and camera remain available for earlier tasks.

PURCHASES NOW

No new mandatory purchases. Do not order a Jetson, graphics card or additional actuators to complete M21. Three short PPO runs with different seeds and BC training fit the established educational limit; achieving a particular reward is not guaranteed.

INSTALL NOW

Add **stable-baselines3 and gymnasium** to a separate M21 environment; use a compatible pinned version of PyTorch CPU from M18. Official SB3 installation: Prerequisites, Stable Release. Save a lock file; the additional Atari and Box2D packages are unnecessary for this task.

System: train the model with the existing simulator in Ubuntu 24.04 ARM64 under UTM; BC analysis can run directly on macOS ARM. When working across systems, record versions, architecture and data format. Do not claim bitwise-identical results across platforms: PyTorch: Reproducibility.

INSTALLATION CHECK

Import torch, gymnasium, stable_baselines3; confirm that the model runs on CPU. Run check_env, reset with a fixed seed, 100 admissible steps and a short PPO update. Check finite values, separate terminated and truncated flags, and policy saving and loading; then confirm that zero residual correction reproduces baseline behaviour.

OPTIONAL, AFTER THE PREREQUISITE IS MET

A second simulator is unnecessary: use previously installed MuJoCo only for an already selected comparison. Consider NVIDIA hardware and Isaac Lab after measuring a CPU shortfall and making a separate G3 decision, scheduled for 27.11-10.12.2028. Isaac Lab: requirements and installation. Apple M4 does not support CUDA; this path is not a prerequisite for M21.

Preparation and checks fit within the existing 75 hours, replacing repeated reading and setup. Delivery waiting time does not count as study time.

Motion fundamentals: the physical meaning of a model action

Required analysis: action, physical effect and time step

Before training, explain the chain: observation → target correction → low-level controller → actuator → mechanical motion → sensor. A normalised network action does not have torque units. In M21, it specifies a roll and pitch correction; the existing controller calculates the force or torque applied. The same correction may produce different torques under different loads.

A minimal model: from cause to state change

For an individual revolute joint, q is the angle in rad and ω is velocity in rad/s. The model $I \cdot d\omega/dt = \tau_{net}$ relates moment of inertia I in kg·m² to net torque τ_{net} in N·m; the latter already includes actuator, gravity and friction contributions with their signs. If I and τ_{net} remain constant over a short interval Δt, then $\Delta\omega = \tau_{net} \cdot \Delta t / I$. Both position and velocity are needed: the same q alone does not determine the next state.

Example. I=0.02 kg·m², τ_{net}=0.10 N·m, Δt=0.02 s, initial ω=0: acceleration is 5 rad/s², final ω=0.10 rad/s, and angle change is 0.001 rad. Double I at the same torque: velocity becomes 0.05 rad/s. This is an analytical physics test; it does not change the policy interface to torque control.

Practice: detect unit and frequency errors

1. Trace the action. In a separate test, normalised a=0.5 with an example scale of 0.04 rad means a target correction of 0.02 rad. Save the raw action, target, applied command and observation. For the actual environment, use its previously verified limit. At zero residual correction, verify agreement with the baseline; under saturation, identify which control level changed the request.

2. Check the integral penalty. For constant error e=0.1 rad and scale e0=0.1 rad, the integrated error penalty over 1 s is the sum $(e/e0)^2 \cdot \Delta t = 1$ s. At Δt=0.02 and 0.01 s, the result is identical; without Δt, the sums become 50 and 100. Repeat on a saved trajectory to separate a metric error from the effect of control frequency on the motion itself.

Acceptance and schedule allocation

Save action_physics.md, a unit table and tests. Explain why doubling mass does not always double every inertia component, and why delay changes closed-loop dynamics. Allocate 1 hour from the analysis in 23-26 and 2 hours from practice in 31-34 to defining actions and rewards. This refines tasks 5-6 within the existing 75 hours; no new training or equipment is required.

Assigned reading: Russ Tedrake, Underactuated Robotics, Definitions - state and equations of motion; Lynch, Park, Modern Robotics §8.9 - gearing and the physical meaning of actuation.